# Build and run a container image with ACR Tasks

In this exercise, you use Azure Container Registry (ACR) Tasks to build and manage container images entirely in the cloud, without requiring a local Docker installation.

Tasks performed in this exercise:

- Download the project starter files
- Deploy Azure Container Registry
- Build and verify container images using ACR Tasks
- Manage image versions and protect production images

This exercise takes approximately **30** minutes to complete.

**Important:** Azure Container Registry task runs are temporarily paused from Azure free credits. This exercise requires a Pay-As-You-Go, or another paid plan.

## Before you start

To complete the exercise, you need:

- An Azure subscription with the permissions to deploy the necessary Azure services. If you don't already have one, you can sign up for one.
- Visual Studio Code on one of the supported platforms.
- The latest version of the Azure CLI.
- Optional: Python 3.12 or greater.

## Download project starter files and deploy Azure services

In this section you download the project starter files and use a script to deploy the necessary services to your Azure subscription. The Azure Container Registry deployment takes a few minutes to complete.

1. Open a browser and enter the following URL to download the starter file. The file will be saved in your default download location.

   Code Copy

   ```
   https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/acr-tasks-python.zip
   ```

2. Copy, or move, the file to a location in your system where you want to work on the project. Then unzip the file into a folder.

3. Launch Visual Studio Code (VS Code) and select **File > Open Folder...** in the menu, then choose the folder containing the project files.

4. Open the *azdeploy.py* deployment script and change the two values at the top of the script to meet your needs, then save your changes. **Note:** Do not change anything else in the script.

   Code Copy

   ```
   "<your-resource-group-name>" # Resource Group name
   "<your-azure-region>" # Azure region for the resources
   ```

5. In the menu bar select **Terminal > New Terminal** to open a terminal window in VS Code.

6. Run the following command to login to your Azure account. Answer the prompts to select your Azure account and subscription for the exercise.

   Code Copy

   ```
   az login
   ```

7. Run the following command to ensure your subscription has the necessary resource provider to install Azure Container Registry (ACR).

```
az provider register --namespace Microsoft.ContainerRegistry
```

8. Make sure you are in the root directory of the project and run the following command in the terminal to launch the deployment script. The deployment script deploys ACR and creates the environment variable files needed for the exercise.

```
python azdeploy.py
```

9. Run the appropriate command to load the environment variables into your terminal session.

**Bash**

```
source .env
```

**PowerShell**

```
. .\.env.ps1
```

**Note:** Keep the terminal open. If you close it and create a new terminal, you might need to run the command to create the environment variable again.

# Build the image with ACR Tasks

In this section you Use a quick task to build the image in Azure without requiring Docker on your local machine. The **az acr build** command uploads your source files, builds the image in the cloud, and pushes it to your registry.

1. Run the following command to build, and push it to your registry. The build completes entirely in Azure. No local Docker installation is required.

**Bash**

```
az acr build \
    --registry $ACR_NAME \
    --image inference-api:v1.0.0 \
    ./api
```

**PowerShell**

```
az acr build `
    --registry $env:ACR_NAME `
    --image inference-api:v1.0.0 `
    ./api
```

2. Watch the output as ACR Tasks:

   - Packs and uploads your source context to Azure
   - Queues and starts the build task
   - Streams the Docker build output showing each layer
   - Pushes the completed image to your registry
   - Reports the image digest and task status

# Verify the image in the registry

In this section you confirm the image exists in your registry by listing repositories and tags.

1. Run the following command to list all repositories in the registry.

   **Bash**

   Code Copy

   ```
   az acr repository list --name $ACR_NAME --output table
   ```

   **PowerShell**

   Code Copy

   ```
   az acr repository list --name $env:ACR_NAME --output table
   ```

   The output shows the **inference-api** repository you created.

2. Run the following command to list tags for the **inference-api** repository.

   **Bash**

   Code Copy

   ```
   az acr repository show-tags \
       --name $ACR_NAME \
       --repository inference-api \
       --output table
   ```

   **PowerShell**

   Code Copy

   ```
   az acr repository show-tags `
       --name $env:ACR_NAME `
       --repository inference-api `
       --output table
   ```

   The output shows the **v1.0.0** tag you assigned during the build.

3. Run the following command to view detailed manifest information, including the digest.

   **Bash**

   Code Copy

   ```
   az acr manifest list-metadata \
       --registry $ACR_NAME \
       --name inference-api \
       --output table
   ```

   **PowerShell**

   Code Copy

   ```
   az acr manifest list-metadata `
       --registry $env:ACR_NAME `
       --name inference-api `
       --output table
   ```

   Note the digest value. This SHA-256 hash uniquely identifies your image regardless of tags.

# Run the image with ACR Tasks

In this section you use the **az acr run** command to execute a command inside your built image and verify it works correctly.

1. Run the following command to verify the Flask application loads correctly in the container.

   **Bash**

Code Copy

```
az acr run \
    --registry $ACR_NAME \
    --cmd "$ACR_NAME.azurecr.io/inference-api:v1.0.0 python -c 'from app import app'" \
    /dev/null
```

**PowerShell**

Code Copy

```
az acr run `
    --registry $env:ACR_NAME `
    --cmd "$env:ACR_NAME.azurecr.io/inference-api:v1.0.0 python -c 'from app import app'" `
    /dev/null
```

The output includes Docker pull progress as it downloads the image. A successful run ends with **Run ID: xxx was successful after xxx**. This confirms the container runs correctly and the Flask application imports without errors.

# Build with a different tag

In this section you build a new version of the image with a different tag to see how the registry maintains multiple versions.

1. Run the following command to build the image again with a new version tag.

   **Bash**

   Code Copy

   ```
   az acr build \
       --registry $ACR_NAME \
       --image inference-api:v1.1.0 \
       ./api
   ```

   **PowerShell**

   Code Copy

   ```
   az acr build `
       --registry $env:ACR_NAME `
       --image inference-api:v1.1.0 `
       ./api
   ```

2. Run the following command to list all tags and see both versions.

   **Bash**

   Code Copy

   ```
   az acr repository show-tags \
       --name $ACR_NAME \
       --repository inference-api \
       --output table
   ```

   **PowerShell**

   Code Copy

   ```
   az acr repository show-tags `
       --name $env:ACR_NAME `
       --repository inference-api `
       --output table
   ```

   Both **v1.0.0** and **v1.1.0** appear in the output, demonstrating how the registry maintains multiple versions.

# View build history and lock a production image

In this section you review the ACR task run history and lock an image to protect it from accidental changes.

1. Run the following command to review the ACR task run history to see all builds you've performed.

**Bash**

Code Copy

```
az acr task list-runs \
    --registry $ACR_NAME \
    --output table
```

**PowerShell**

Code Copy

```
az acr task list-runs `
    --registry $env:ACR_NAME `
    --output table
```

The output shows each build task with its run ID, status, trigger type, and duration. This history helps you track builds and diagnose issues.

2. Run the following command to lock your v1.0.0 image to prevent accidental deletion or modification.

**Bash**

Code Copy

```
az acr repository update \
    --name $ACR_NAME \
    --image inference-api:v1.0.0 \
    --write-enabled false
```

**PowerShell**

Code Copy

```
az acr repository update `
    --name $env:ACR_NAME `
    --image inference-api:v1.0.0 `
    --write-enabled false
```

3. Run the following command to verify the lock is in place.

**Bash**

Code Copy

```
az acr repository show \
    --name $ACR_NAME \
    --image inference-api:v1.0.0
```

**PowerShell**

Code Copy

```
az acr repository show `
    --name $env:ACR_NAME `
    --image inference-api:v1.0.0
```

The **writeEnabled** field shows **False**, indicating the image is protected.

# Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. Run the following command in the VS Code terminal to delete the resource group, and all resources in the group. Replace **<rg-name>** with the name you choose earlier in the exercise. The command will launch a background task in Azure to delete the resource group.

Code Copy

```
az group delete --name <rg-name> --no-wait --yes
```

**CAUTION:** Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.

# Troubleshooting

If you encounter issues while completing this exercise, try the following troubleshooting steps:

### Verify Azure authentication and environment variables

- Run **az account show** to confirm you're logged in to the correct Azure subscription.
- Verify your environment variables are set by running **echo $ACR_NAME** (Bash) or **$env:ACR_NAME** (PowerShell).
- If variables are empty, re-run **source .env** (Bash) or **. ..env.ps1** (PowerShell).

### Verify ACR deployment

- Navigate to the Azure portal and locate your resource group.
- Confirm that the Azure Container Registry exists and shows a **Provisioning State** of **Succeeded**.
- Run **az acr list --output table** to verify your registry is accessible.

### Troubleshoot build failures

- Check the build output for error messages - common issues include missing Dockerfile or incorrect file paths.
- Verify you're running commands from the project root directory (where the *api* folder is located).
- Run **az acr task list-runs --registry $ACR_NAME --output table** to see the status of recent builds.